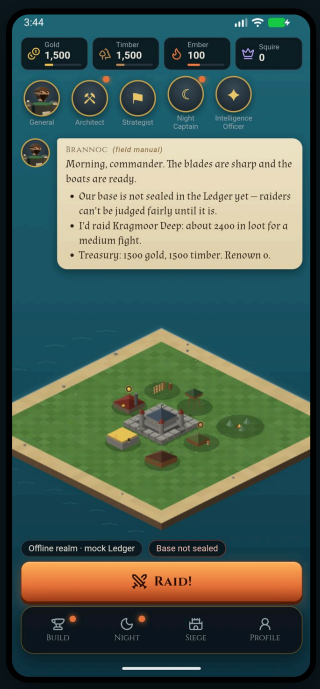


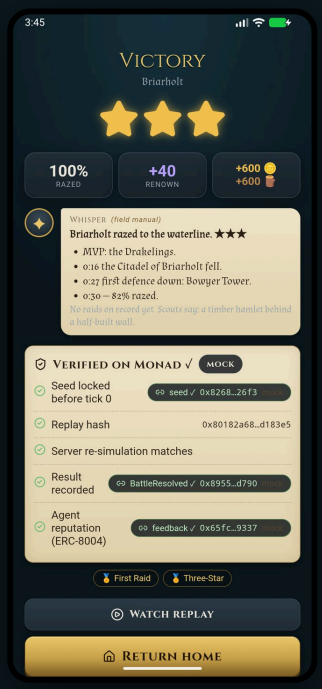
PROJECT REPORT

Realm Raiders

What was built, how it works, and how it was made







START HERE

The one-page version

Realm Raiders is a strategy game for Android phones. You build a castle on an island, create a 3D General, and raid other islands. Five AI "officers" advise you. The result of every raid is written to the **Monad** blockchain, so anyone can check that nobody cheated.

17
landscape screens, from sign-in to a Live Duel with a friend

5
AI officers running on a local Qwen3 model

4 + 2
smart contracts live and verified on Monad testnet, plus the 2 official ERC-8004 registries

≈33K
lines of code in 250 source files

208
automated tests passing, plus 20 browser play-throughs and 3 Android runs

22 / 22
real Monad transactions confirmed "success"

12–24 ms
to simulate a whole 3-minute battle

5.7 MB
signed release APK (debug build: 7.1 MB)

What was built

- **The phone app**: 17 landscape screens with a base-builder HUD, an isometric island in a code-drawn forest with pinch zoom, troops and buildings modelled in 3D and baked into sprites on the phone, a 3D General creator, synthesised sound effects and original background music. All art is made by code; none of it was downloaded.
- **Gems and Auto Mode**: an in-game currency earned only by playing, spent on letting the officers run the realm under orders sealed on chain — every move checked and logged.
- **Play with Friends**: on the same Wi-Fi, raid a friend while they watch live, fight a Live Duel on one shared seed (recorded on the new *RealmDuels* contract).
- **The battle engine**: the rulebook for every fight. It gives exactly the same result on any phone or computer, which is what makes raids provable.
- **The game server**: stores players and bases, re-checks every battle, runs the AI officers, and sends the blockchain transactions.
- **Five AI officers**: General, Architect, Strategist, Night Captain and Intelligence Officer. They speak through a Qwen language model, but every answer is checked. If an answer fails a check, the officer uses its built-in rules instead.
- **Smart contracts on Monad**: *RealmLedger* (bases, seeds, results, night and Auto orders), *RealmBadges* (achievements) and *RealmDuels* (Live Duel results). Each General is also registered as an **ERC-8004 agent** with an on-chain reputation.
- **The rest**: an **Android app** package, a **landing website**, and a **replay-checking tool** anyone can run.

Contents

1 The game in simple words

2 Architecture: the big picture

3 How the code is organised

4 The battle engine

5 How a raid is proven

6 The smart contracts on Monad

7 The five AI officers

8 Night defence

9 The phone app, screen by screen

10 The 3D General, art and sound

11 Online, offline and Android

12 The landing website and tools

13 How it was built

14 Proof that it works

15 What is not done yet

16 How to run it and glossary

WHAT'S NEW

The enhancement, in eight phases and a play-test

After the first release the game was rebuilt in eight checked phases, each with its own tests and report (`docs/PHASE_REPORTS.md`). The rules never changed underneath old battles: the battle engine gained a **version 2**, and every recorded raid still re-simulates under the rules it was fought with.

Phase	What changed
1 Bugs	Wall lines that place all-or-nothing with the exact reason; zoom and drag that never get swallowed; a Profile that loads fast and shares an image.
2 Camera	Pinch and wheel zoom about the finger; drag to pan with a fling; long-press to pick a building up.
3 Landscape	A base-builder HUD in landscape, bevelled buttons, bundled fonts, and an island in a forest drawn by code (about 730 batched trees).
4 Troops	Swordsman, Archer, Stone Giant, Ember Mage, Fire Dragon (and the Sapper and General) with real counters — engine v2, integer maths, the old rules frozen as v1.
5 3D	Every troop and building modelled in Three.js, baked on the phone into 5-direction × 22-frame sprite sheets ("Forging your army…", about 1 s), cached; fire, fireball, slam and arrow effects.
6 Gems + Auto	Gems earned only by playing; Auto Mode lets the officers collect, build, raid and defend under sealed Auto Orders, every move logged on chain.
7 Friends	`pnpm dev:lan` hosts a room on a laptop; phones find it on the Wi-Fi; raid a friend while they watch live; Live Duels on one shared seed recorded on *RealmDuels*.
8 Polish	Synthesised sounds for every new attack, tutorial tips, the signed release APK, and this document.
Play-test pass	Wall lines painted by dragging, with corners filled in; a Shop with pictures where every Citadel-1 building can be bought and placed (a ghost, then ✓); a calmer, symmetric home screen in plain words; pinch that hands over cleanly to a drag; no white flashing on hit buildings; a bigger Fire Dragon with a shadow and breath from its mouth; full-screen target scouting; original background music. The Clan Siege was removed.

Old proofs stay true

A replay records which rules it was fought under. The v1 rules are frozen in code, and the server fixes a battle's version when it is committed, so a v1 army can never be scored against a v2 seed. The stability fingerprints (200 seeded battles hashed together) are pinned for both versions and checked in Node, Chrome and the Android WebView:

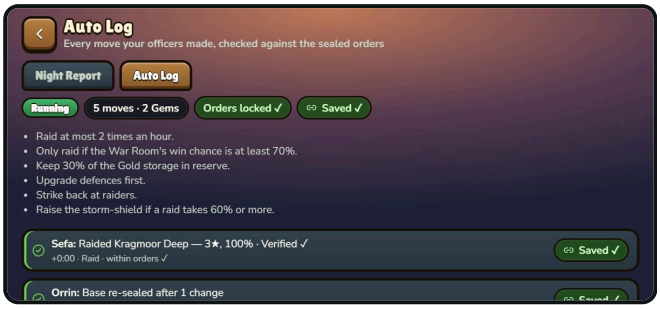
Rules	Fingerprint	Goldens
v1 (launch, frozen)	`0xc0afd396…1037ebe`	3 replays, byte-identical since launch
v2 (current)	`0xebb7fe98…65330d5dc6`	3 new replays (sword rush, giants + sappers, dragon)

Auto Mode: orders, not trust

The officers act on a fixed 5-minute beat. Before anything happens, each move is checked against the sealed Auto Orders (raids per hour, minimum win chance, Gold reserve, what to upgrade first, strike back, storm-shield) and logged with `logAutopilot` under the session's own on-chain id. A refused move is written to the Auto Log with its reason — "Refused: upgrade Citadel to level 2 (would leave 1500 Gold, orders keep 1950)" — and nothing happens. A closed offline realm catches up on exactly the beats it missed.



Auto Mode running
The officers are in charge; manual play waits for TAKE CONTROL.



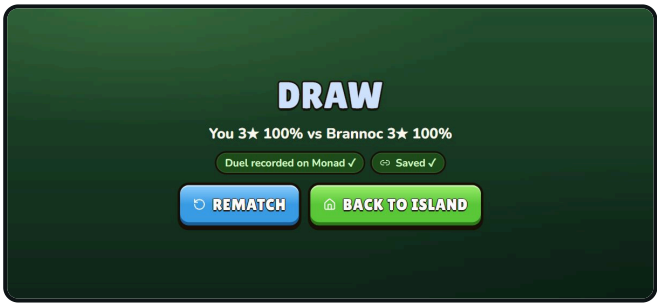
The Auto Log
Collected, upgraded, re-sealed, raided — each with its Ledger entry.

Play with Friends: the replay is the battle

When you raid a friend, your phone streams the actions it applied every half second. Their phone runs the same deterministic battle a moment behind — so what they watch is exactly what the host then verifies. In a Live Duel, one commit-reveal locks a single seed for both commanders; both replays are re-simulated, the winner is decided by stars, then %, then time, and the result goes on chain (*RealmDuels*, verified on Sourcify).



Live Duel
"You 2★ 64% · Brannoc 1★ 48%" updates every 500 ms.



Duel recorded on Monad ✓
Both replays re-simulated by the host.

CHAPTER 1

The game in simple words

The game is set in the **Emberwake Isles**, an original world: a cold northern sea of storm-cut islands, left behind when a sky-dragon fell and broke the old continent. Every lord wants *ember*, the glowing stuff that washes up on the shores.

You are a lord with one island. During the **day** you build and raid. At **night** you give orders and go to sleep, and your AI officers defend the island within those orders. In the morning you read what happened.

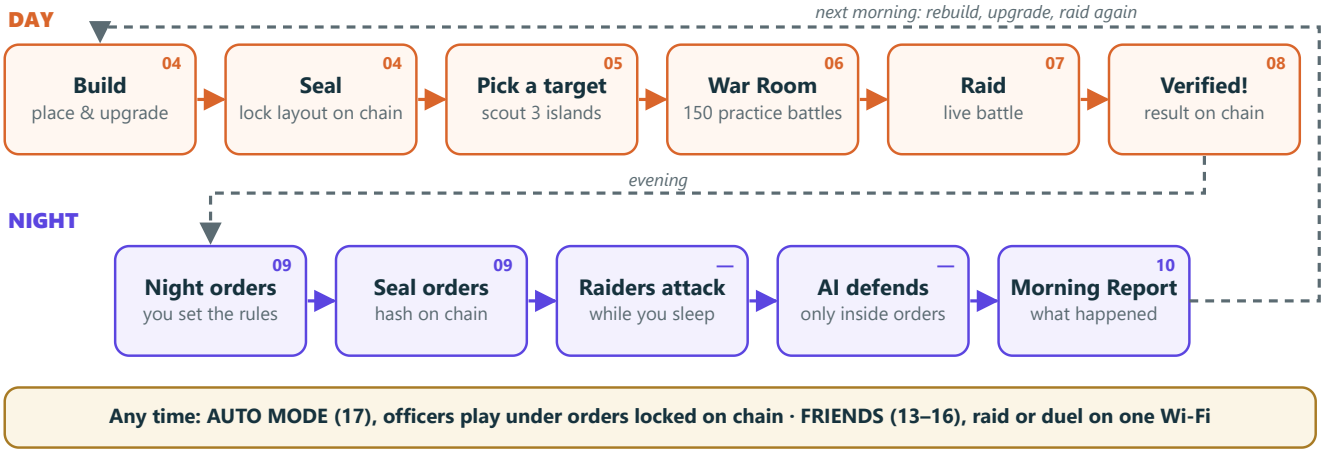


Figure 1. The game loop. Small numbers in the corners are the screen numbers in chapter 9.

The pieces of the game

Thing	What it is
Resources	**Gold** and **Timber** for building, **Ember** for the rare troops. Mines and camps produce them over time; you tap a building to collect.
Buildings (14)	Citadel (your main keep), Goldworks, Treasury, Lumber Camp, Timber Yard, Ember Well, Muster Hall, War Camp, the defences (Bowyer Tower, Stonethrower, Rune Obelisk, Wyrm Ballista), Stone Wall and Caltrop Pit (a trap).
Troops (7)	Swordsman (cheap front line), Archer (shoots over walls), Sapper (breaks walls), Stone Giant (goes for defences first), Ember Mage (a fan of three fireballs), Fire Dragon (flies, breathes a fire cone), and your **General**, a hero with a **Rally** power.
Gems	Earned only by playing (daily login, first raid of the day, 3-star raids, badges, duel wins) and spent on Auto Mode. Gems can't be bought.
Stars	A raid earns up to 3 stars: one for destroying half the island, one for the Citadel, and one for destroying all of it.
Renown and ranks	Winning gives Renown. Ranks go Squire → Knight (100) → Banneret (300) → Warden (700) → High Marshal (1,500).
Rivals	Three computer-run islands to raid: **Briarholt** (easy), **Kragmoor Deep** (medium) and **Saltspire Eyrie** (hard). Their lords also raid you at night.
Badges	First Seal, First Raid, Three-Star, Night Watch and Architect's Pupil. They are *soulbound* tokens on the chain: they can't be sold or moved.



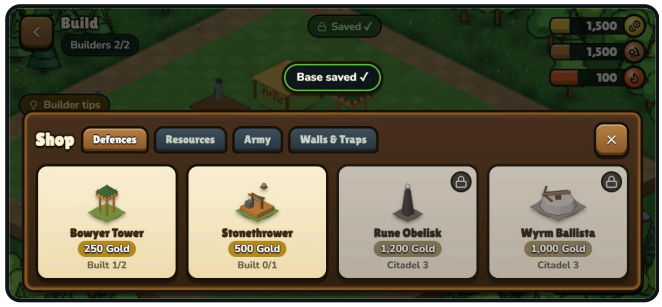
Your island
Resources top-right, advisers on the left, Auto on the right, ATTACK and the Shop in the corners.



A raid
The seed is "locked" on the chain before the fight starts.



The result
Stars, then a checklist proving the fight was fair.



The Shop
A picture of every building, its price and how many you can still build.

What makes it different

- **Every raid is provable.** You don't have to trust the game's server; anyone can re-run a raid and compare it with the public record.
- **AI officers with real jobs.** They plan raids by actually simulating them, check your walls, and write your reports, and they can't invent numbers.
- **The AI acts while you sleep**, but only inside orders you sealed. Each action is logged against those orders.
- **No crypto knowledge needed.** No wallet to set up, no gas to pay.
- **Works offline.** With no server around, the whole game runs on the phone.

A new player's first ten minutes

1. Tap **Guest commander** (or Google/email when set up).
2. Style a 3D **General**, give them a name, and raise them. They get an on-chain agent ID.
3. Land on the **home island**. The General says what to do first.
4. Open **Build** and tap **SAVE BASE**. The layout's fingerprint goes on chain, so raids on you can be judged fairly.
5. Tap **ATTACK**, swipe to Briarholt, and let the Strategist test three plans.
6. Tap **ATTACK!** (the battle seed is locked on chain), watch or take control, and get a "Fair play verified ✓" result.
7. In the evening, save **night orders**. Next morning, read the **report**.
8. Turn on **Auto Mode** and let the officers play inside the orders you locked.

CHAPTER 2

ARCHITECTURE: THE BIG PICTURE

The system has four big parts: the **phone**, the **server**, the **AI model** and the **blockchain**. Under them is a set of **shared packages** that run on both the phone and the server. That shared code is the key idea of the design.

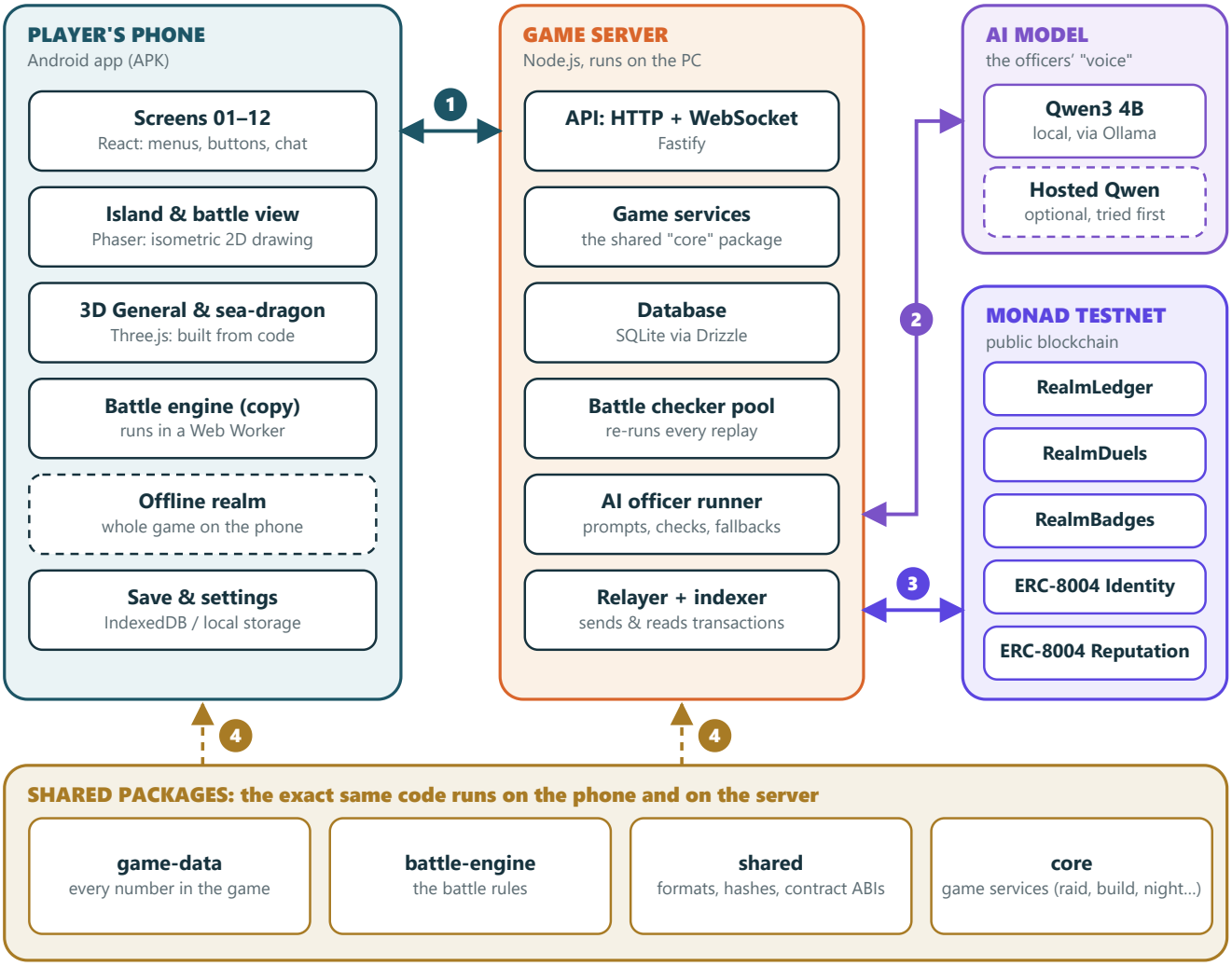


Figure 2. The whole system. Each numbered link is explained below.

1. **Phone ↔ server.** The app asks for data (your island, targets, reports) and sends its battle replays. The server pushes live updates over a *WebSocket* (Auto Mode moves, friend raids, duels, night raids, officer messages).
2. **Server ↔ AI model.** The officer runner sends a question plus the true game facts. The model replies in a fixed JSON format, and the runner checks the reply before anyone sees it.
3. **Server ↔ Monad.** A *relayer* wallet on the server sends the transactions and pays the gas, so players never need crypto. An *indexer* reads the events back.
4. **Shared packages.** The same game data, battle rules and game services are built into both the app and the server, so the two always agree.

IN SIMPLE WORDS

Think of a football match with a referee. The **battle engine** is the rulebook, and the phone and the server each have an identical copy. Your phone plays the match. The server, as referee, replays it from your recording using the same rulebook. If the two copies of the rules agree, the result is stamped into a public notebook (the **Monad blockchain**) where nobody can change it. The **AI officers** are coaches who give advice, but they can only use numbers that really exist.

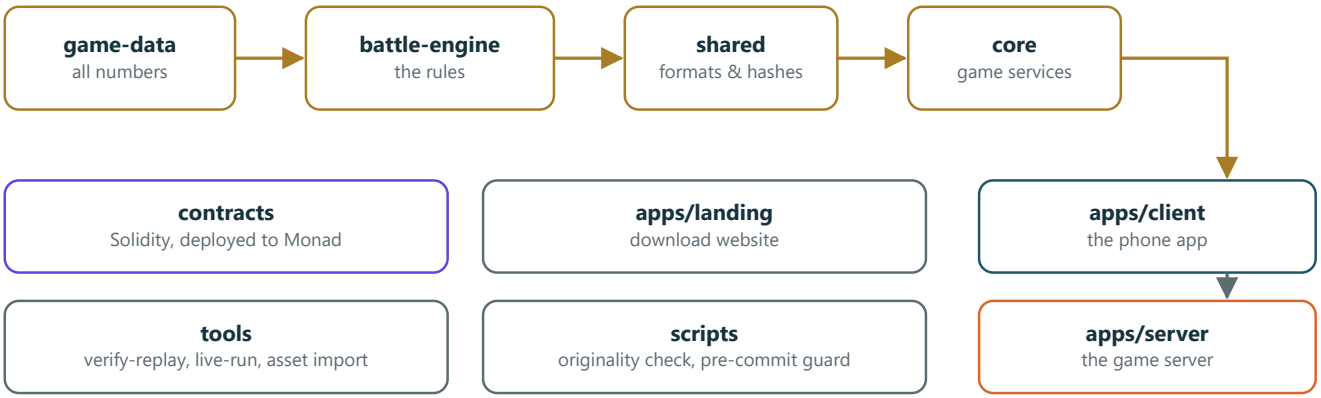
Why it is built this way

- **Fair raids without trusting anyone.** Battles run on the phone, so they are smooth and work offline. Because the engine is *deterministic* (chapter 4), the server can re-run any battle and catch a faked result.
- **One codebase, two places.** The game services (`packages/core`) run on the server *and* inside the app. With no server in reach, the whole game still runs on the phone.
- **No crypto knowledge needed.** Players never see a wallet or pay gas. The server's relayer pays for the transactions, and players see a simple "Saved ✓" chip (on the live chain: "Monad ✓", which opens the explorer).
- **AI that can't break the game.** Officers give advice and draft orders, but every answer is checked, and actions on your behalf must fit orders you sealed on chain (chapter 8).

CHAPTER 3

How the code is organised

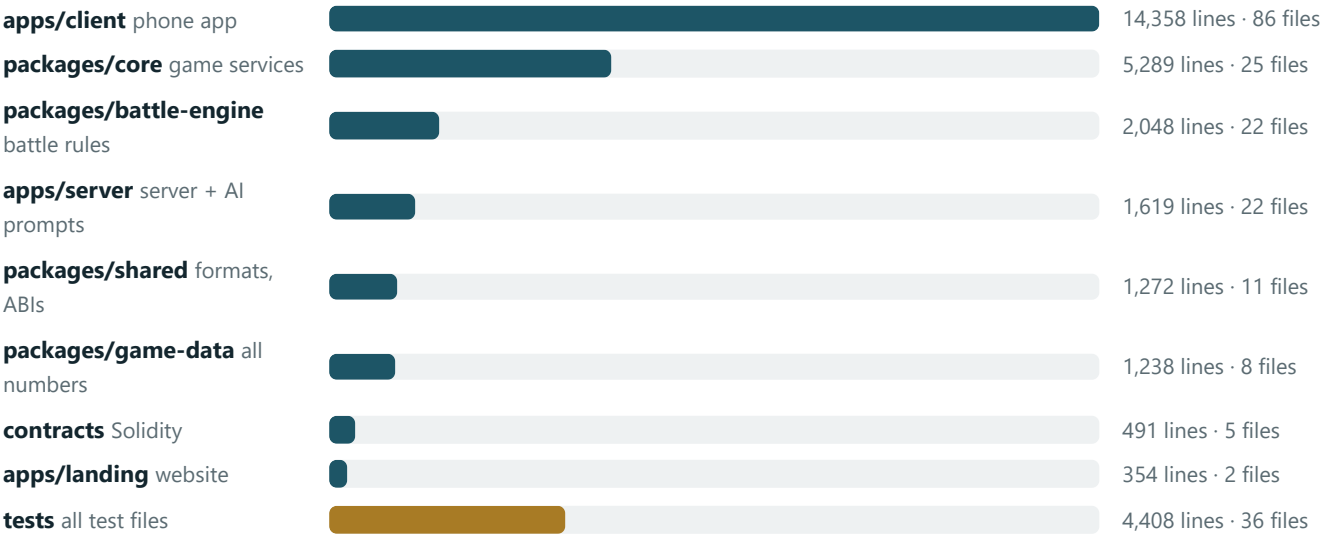
Everything lives in one folder, `realm-raiders/`, as a **monorepo**: one repository holding several small packages that use each other. The **pnpm** package manager links them. All the code is **TypeScript** (JavaScript with types), except the contracts, which are **Solidity**.



Arrows read "is used by". The app and the server both build on the same four packages.

Figure 3. Package map. Data flows left to right: numbers, then rules, then formats, then game services, then the app and the server.

Size of each part



Folder	What is inside, in plain words
`packages/game-data`	Every number in the game: building and troop stats, costs, the economy, ranks, badges and the three rival islands. If you want to rebalance the game, this is the only place to edit.
`packages/battle-engine`	The battle rules: movement, path-finding, targeting, damage, traps, the General's rally, stars. It also handles replays (record and re-play), the replay hash, and the verifier. It uses integer maths only (chapter 4).
`packages/shared`	Agreements between the app, the server and the chain: request and response shapes, how bases are turned into bytes and hashed, the contract interfaces (ABIs), the night-order rules, and the Monad chain settings.
`packages/core`	The game services, one per feature: home, General, base, targets, War Room, battle, night, Auto Mode, Play with Friends, profile. It also holds a rule-based "field manual" for each officer and a mock blockchain for offline play.
`apps/server`	The **Fastify** web server: API routes, WebSocket hub, **SQLite** database, login sessions, a pool of worker threads that re-check battles, the AI officer runner with its prompts, and the **viem** relayer and indexer for Monad.
`apps/client`	The phone app: **React** screens 01–12, **Phaser** scenes for the island and battle, **Three.js** for the 3D General and the baked troops, sound effects and music, and the **Capacitor** Android project.
`apps/landing`	The download website, with a 3D island at the top and the Android "App Links" file.
`contracts`	Solidity contracts, their tests, and scripts to compile, deploy and verify them.
`tools`, `scripts`	`verify-replay` (anyone can re-check a raid), `live-run` (plays every on-chain moment end to end), an asset importer, an originality check, and a pre-commit guard that blocks secrets from being committed.

CHAPTER 4

The battle engine

The engine is the heart of the game. Give it an island, an army, a seed and the list of your taps, and it plays out the whole battle, tick by tick. It runs inside the app (in a background *Web Worker*, so the screen never freezes) and on the server (in a pool of worker threads).

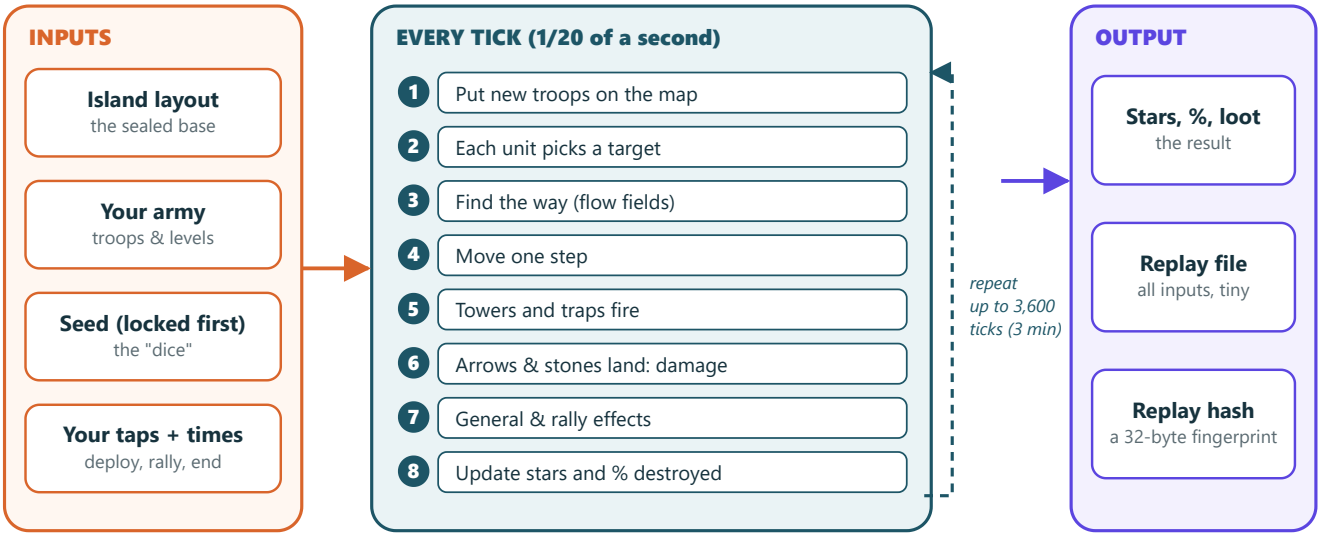


Figure 4. One battle. 20 ticks per second, at most 3 minutes. The output is the result, a small replay file, and its fingerprint (hash).

The one rule: same input, same answer, everywhere

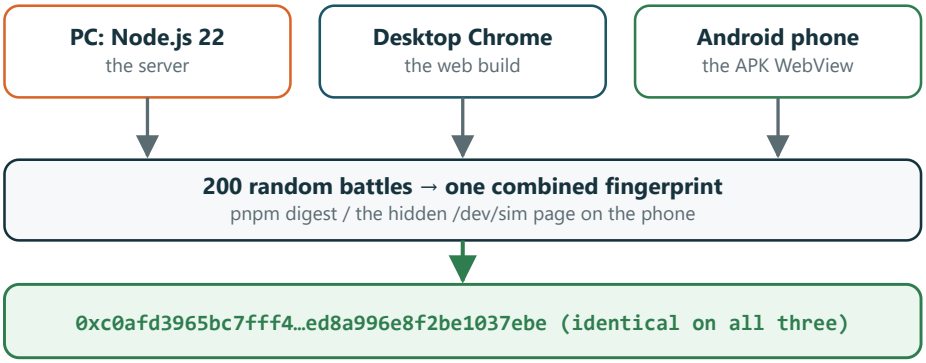
> **IN SIMPLE WORDS**
>
> Different phones can round decimal numbers slightly differently. Over thousands of steps, a tiny difference grows until one phone says "3 stars" and another says "2 stars". To prevent this, the engine never uses decimals. Distances, health and timers are all **whole numbers**: one tile is 1,000 units, and health is counted in thousandths of a point. Then every device gets exactly the same answer, like a recipe followed to the gram.

- **Its own dice.** Randomness comes from a small built-in generator called *sfc32*, started from the battle's seed. Same seed, same "random" events.
- **No risky maths.** No square roots, sines or decimal division. The engine has its own integer square root and a 16-direction compass. A test scans the engine's source code and fails if anyone uses `Math.random`, `Math.sqrt`, `Date` and similar functions.
- **Smart path-finding.** Troops find their way with *flow fields*: picture water flowing downhill toward every target at once. Walls count as "expensive" ground, so troops go around them unless breaking through is shorter.
- **Fingerprints.** The replay is hashed with *keccak256*, the same hash Ethereum and Monad use, so the contract can store it and anyone can compare.

Proof that it really is identical



/DEV/SIM
A hidden page (tap the sign-in crest 7 times) that runs 200 random battles and 3 recorded "golden" battles on the device itself.



The engine plays **200 battles with random seeds** and combines all their results into one fingerprint. That fingerprint is stored in a test. It came out **exactly the same** in Node.js on the PC, in desktop Chrome, and inside the real Android app on the emulator (Android 17, Chrome 145 WebView).

Three recorded **golden battles** also replay to the same hash, with check-hashes every 100 ticks along the way. If someone changes the rules by accident, these tests fail at once.

Speed: a full battle takes about **12–24 ms** to simulate. The War Room runs 3 plans × 50 battles = **150 practice battles in about 1.3 seconds** on the server.

CHAPTER 5

How a raid is proven

A cheater could try two things: **pick a lucky seed** (re-rolling the dice until the battle goes their way), or **send a fake result**. The raid flow below blocks both.

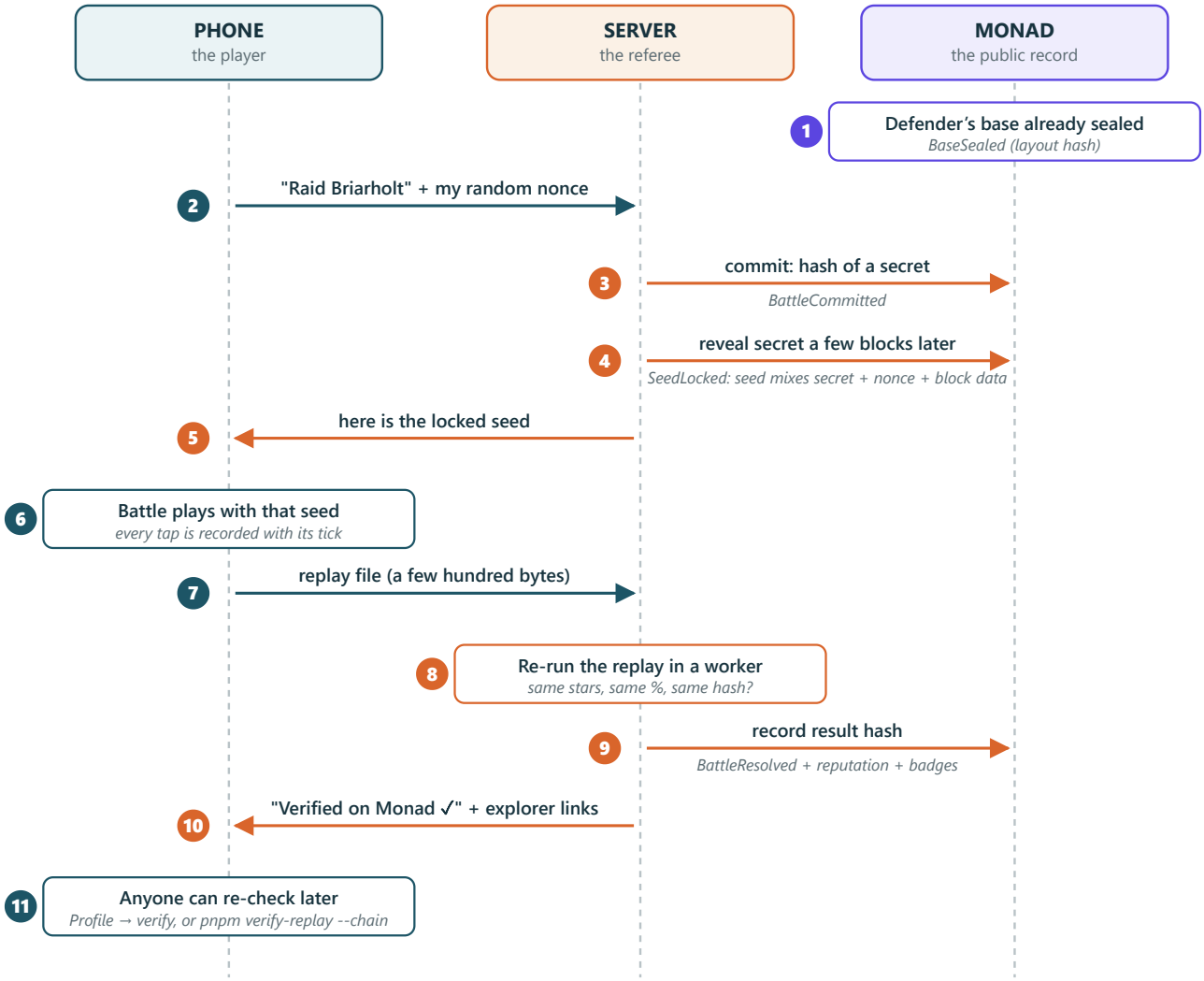


Figure 5. A raid, step by step. Grey dashed lines are the three actors. Labels in italics are the events written to the Monad blockchain.

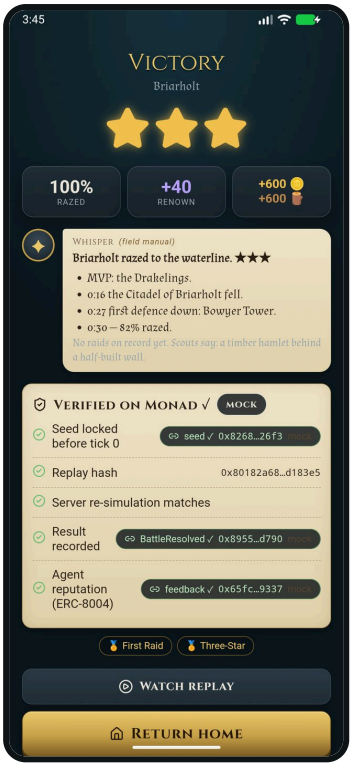
IN SIMPLE WORDS

The seed is a dice roll nobody can secretly choose. First the server puts a secret in a "sealed envelope" by publishing only its hash on the chain (*commit*). A few blocks later it opens the envelope (*reveal*). The final seed mixes that secret with a random number from the player and data from the blockchain block itself. The server can't change its secret after committing, and the player can't predict the block data. So neither side can pick a lucky seed alone.

The result can't be faked. The phone sends a **replay**, the list of what you tapped and when, not the result. The server re-runs it with the same engine. Only if the stars, percentage and hash match does it record the result on chain. The replay also carries a copy of the defender's base, checked against the hash that was sealed on chain before the raid.



Before
The War Room shows the target's base "Sealed in the Ledger ✓" before you can lock the seed.



After
"Verified on Monad": seed locked, replay hash, server re-simulation, result recorded, reputation.

Anyone can check, not just the server

- On the **Profile** screen, the *verify* button downloads a battle's replay and re-runs it **on your phone**.
- On a computer, `pnpm verify-replay <battleId> --chain` re-simulates the replay and checks it against what RealmLedger stored for that battle: the **seed**, the **base hash**, the **replay hash**, the **stars** and the **destruction %**. A live raid passed **all 5**.

> In this screenshot the chips say "mock" because the phone was playing offline. In live mode the same chips link to real Monad transactions (chapter 6).

CHAPTER 6

The smart contracts on Monad

A **smart contract** is a small program stored on a blockchain. Once deployed, its code and data are public and nobody can quietly change them. Realm Raiders uses four of its own contracts plus the two official **ERC-8004** registries, a shared standard for giving AI agents an on-chain identity and reputation.

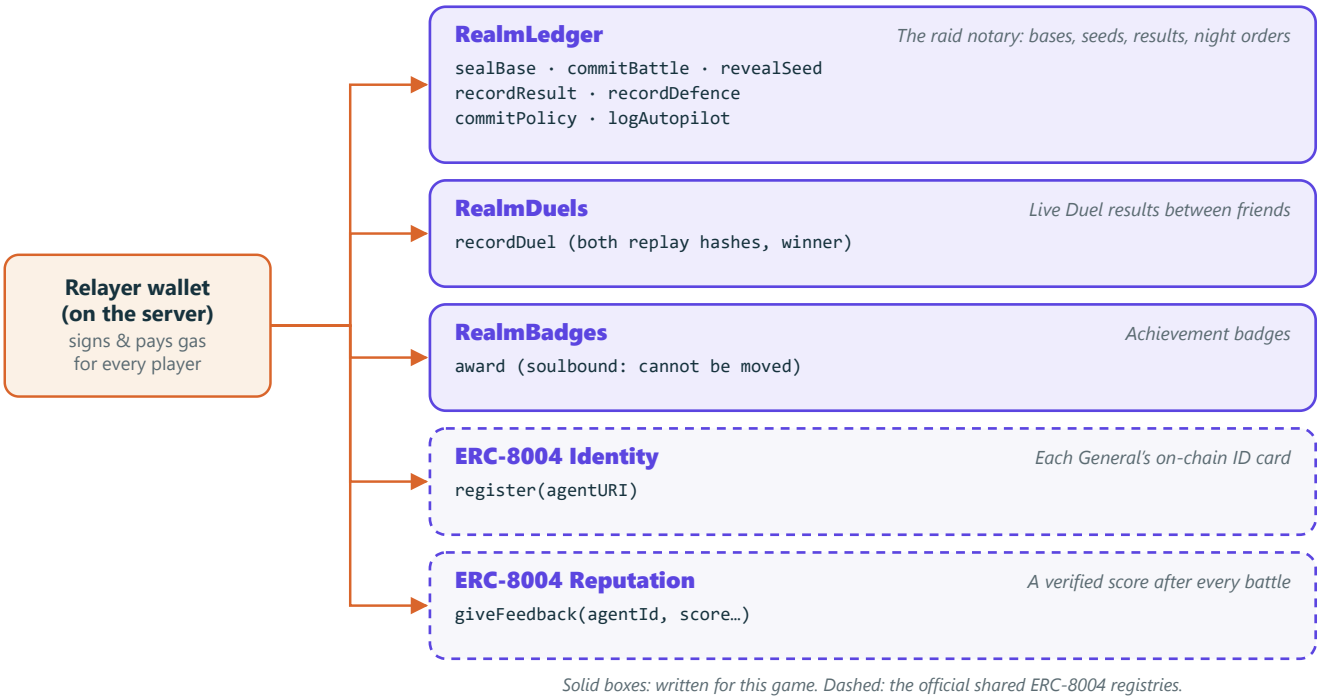


Figure 6. Who calls what. The relayer wallet on the server sends every transaction and pays the gas. Where a contract needs the player's own consent, such as sealing a base or committing night orders, the player's signature travels with the transaction (`sealBaseFor`, `commitPolicyFor`).

Deployed and verified

The contracts were deployed to **Monad testnet** (chain 10143) on 3 November 2026, and **RealmDuels** on 4 November 2026. Their source code is **verified** on Sourcify (the verifier behind the MonadVision explorer): anyone can read the exact Solidity that runs at each address. The data below was read live from the chain when this document was made (04 Nov 2026 05:26 UTC, chain head block 76,833,310).

Contract	Address	Code size	Source verified
RealmLedger	`0xec0e5ae643491f288f0332f695dc2f20fdbd0352`	8,835 bytes	**perfect match ✓**
RealmBadges	`0xfe87d11da421427f0a72f587efc7dcc13f0d4d70`	6,459 bytes	**perfect match ✓**
RealmDuels	`0x069d80675b73da9971620ac06c146e87262af3ca`	3,480 bytes	**perfect match ✓**

How they were written and tested

- Written in **Solidity 0.8.38** on top of **OpenZeppelin 5.6.1**, a widely used library for roles, tokens and signatures.
- Compiled with **solc-js** and tested on an **in-memory Ethereum machine** (*@ethereumjs/vm*). 9 tests cover sealing, the commit–reveal seed, results, defences, night orders, autopilot rules, badges, duel results and the retired siege contract.
- Deployed with a **viem** script, then verified with a Sourcify script. The usual Foundry toolkit couldn't be installed on this PC, so this route replaced it (chapter 15).
- Badges use **ERC-1155** with transfers switched off ("soulbound").

Real transactions on Monad testnet

The `tools/live-run.ts` script played every on-chain moment of the game through the real server and the real chain. Each row below was read back from the Monad RPC: all 15 transactions are confirmed with status **success**.

What happened in the game	Contract	Block	Gas	Status	Transaction
General registered as ERC-8004 agent	ERC-8004 Identity	76,829,418	258,338	**success**	`0x5af31284bba9350fd9946de8f10fb4904ee76dadc8cbc6daa81c09d51869af6a`
BaseSealed (layout hash stored)	RealmLedger	76,829,431	134,164	**success**	`0x82039c59d4ed26ebf42acedbdd3acdec062c25677e9a4f2667877e181c829a17`
BattleCommitted (secret seed committed)	RealmLedger	76,829,478	203,579	**success**	`0x454e6cb770814b4c5890c58e97ac135735c68792bd5abe6b3a5b9ac04aa1bbe8`
SeedLocked (seed revealed + mixed)	RealmLedger	76,829,485	71,518	**success**	`0x2f7d1000377f47cfbba8d9116543e938c4f1925bab06e6d9ae7629b4a348871d`
BattleResolved (result hash stored)	RealmLedger	76,829,491	106,714	**success**	`0x155754285d8e742793efa421eaad3b57cbfec111b7a2db346ba19ca722a62c1b`
ERC-8004 reputation feedback	ERC-8004 Reputation	76,829,495	342,306	**success**	`0xda4aa974353de2ef142597176d577625ce1ac5a0ecfe782c35076ad795736b8b`
Badge minted: First Raid	RealmBadges	76,829,500	84,303	**success**	`0x07ef20d40d926cb4865d76a25308363c94df695d550fa39695d56d6b425cee7f`
Badge minted: Three-Star	RealmBadges	76,829,506	84,303	**success**	`0x7a9570a2e19b79a03412db210f59329bcd0610533379f26a16e143545c3c9d71`
PolicyCommitted (night orders sealed)	RealmLedger	76,829,512	115,883	**success**	`0x25ff36bd42e4f3ed7f766b9b6b6fc99984e8d8ad0958bfbf7c11f031da3381e3`
DefenseRecorded vs Briarholt (0 stars)	RealmLedger	76,829,529	87,019	**success**	`0xd47838a16a0572ae8ddd47e27d6fb7206abb95ff948e998c4c3327949f0afe59`
DefenseRecorded vs Kragmoor (0 stars)	RealmLedger	76,829,556	87,019	**success**	`0x6eadecbd36cbd3b66de5a255d814d2c50d4ed81ff6c5814d9e8b7adbfde0cab3`
DefenseRecorded vs Saltspire (0 stars)	RealmLedger	76,829,578	87,005	**success**	`0xa53a47622e6f3b30a4a4b9978acb4d01a001338f44bb80418c5d0bb1ca1a995d`
AutopilotAction: repaired walls	RealmLedger	76,829,534	49,443	**success**	`0xb1cb4606f1f45ec8a42c3bfcbca517903942754b41cef1e453ced1ed529beaa7`
AutopilotAction: struck back at the raiders	RealmLedger	76,829,541	49,443	**success**	`0xe0e666a0b34570de3c5cfa9bdbb51514a7128e7864619898db1f41f7eff7f10e`
AutopilotAction: repaired walls	RealmLedger	76,829,562	49,443	**success**	`0x2844017d2093ad52e3220e04f778a5f8965bbee75a66135b381fcc79494d4997`

What happened in the game	Contract	Block	Gas	Status	Transaction
AutopilotAction: repaired walls	RealmLedger	76,829,584	49,443	success	0x2e9c887341575e2b7294c4065b6605288d3aaef242d0e1ff7ab6ae8519755941
PolicyCommitted (Auto Orders locked)	RealmLedger	76,829,596	115,869	success	0x9055faea05ef05c23b3540a382f0f0e9deacb5ab8f7bf46211b30760adddd264
Auto: collected Gold and Timber	RealmLedger	76,829,601	49,443	success	0xf22b044fcbac693c6c97efc4049aaffe873e093932f7733876f274a3e7512dc4
Auto: built a new Goldworks	RealmLedger	76,829,606	49,443	success	0xff0675fd02f40b485bc4e3e778cb35f5d1254122ca51e816a610476922f6b819
Auto: moved 2 buildings (Architect's fix)	RealmLedger	76,829,613	49,443	success	0xa5a4eb8928d411827091467c9a0bf06e15dc7f6bfb59b35dd3ac4ac18e111077
Auto: base re-sealed	RealmLedger	76,829,622	49,443	success	0x7b68d0a5e9bf08f8a026719c212acc2890510cb3a72a49f8bf245864e0a7bee0
Auto: raided Kragmoor Deep, 3 stars, verified	RealmLedger	76,829,647	49,443	success	0xb666ead3db907d4853ce38da9c5203b94cea51a183b3c95293ec565637d38a9d

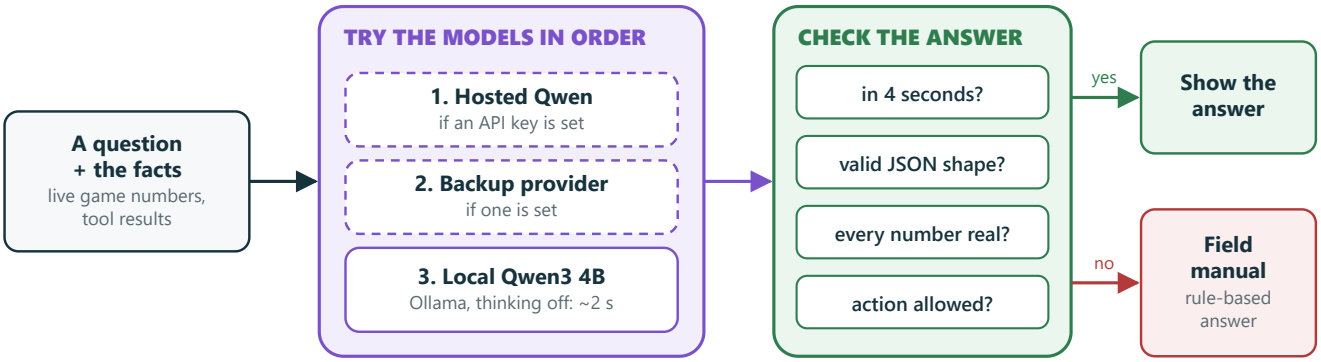
Each transaction can be opened at `testnet.monadvision.com/tx/<hash>`. Later, the real APK on the emulator also played a live raid: its General became agent **#2563**, the reputation registry showed 1 verified battle with an average score of 100.

CHAPTER 7

The five AI officers

Each player has a council of five officers. They speak in character and give useful advice, but they are never trusted blindly.

Officer	What they do	Where you meet them
Your General the name you chose	Greets you each morning with a short briefing (base sealed? best target? treasury) and speaks on your profile. Their personality (Bold, Cunning, Steadfast or Merry) changes the tone.	02 creator, 03 home, 12 profile
Master Mason Orrin Architect	Studies your layout ("2 stores sit outside the walls") and can propose a fix that moves buildings for you.	04 build, 10 report
Captain Sefa Brightspear Strategist	Runs 50 practice battles for each of 3 plans, recommends one, and takes your coaching ("more sappers"). Calls out live tips during the raid.	06 War Room, 07 battle
Watch-Captain Hale Night Captain	Drafts your night orders: what to repair first, how much gold to keep in reserve, when to strike back, and when to raise the storm-shield.	09 night orders
Whisper Intelligence Officer	Scouting notes on targets, a battle report after each raid (MVP, key moments), and the Morning Report summary.	05 targets, 08 result, 10 report



One repair retry on bad JSON. Answers are cached and rate-limited. The officer is tagged "(field manual)" when the rules answered.

Figure 7. How an officer answers. The model only supplies the wording. The facts come from the game, and a failed check falls back to rules.

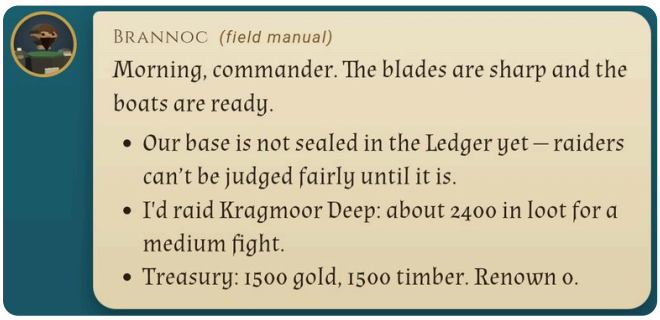
> **IN SIMPLE WORDS**
>
> Language models sometimes invent things. So each officer gets the **real facts** (your gold, the win rates from the practice battles, and so on) and must reply in a fixed format. A check then looks at **every number** in the reply. If a number doesn't appear in the facts, the reply is thrown away and the officer answers from its **field manual**, a set of written rules that always works. The screen tags those answers "(field manual)", so you always know which kind you got.

Which model, and how fast

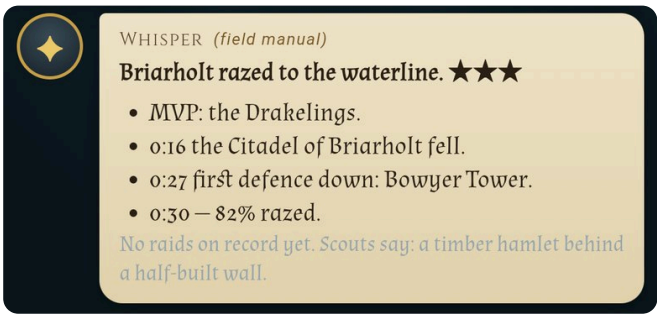
- The live officers use **Qwen3 4B** running locally through **Ollama**, so no API keys or cloud costs. A hosted Qwen can be plugged in with two settings and is tried first.
- Qwen3 normally "thinks out loud" before answering, which took about 10 seconds. Switching that off (`reasoning_effort: "none"`) and using the 4B model brought answers to **1.8–3.2 seconds**, inside the 4-second budget.
- The greeting, the briefing, the night draft and the War Room advice were answered live by the model during testing. The other officers fall back cleanly when the model is slow.

The officers speaking

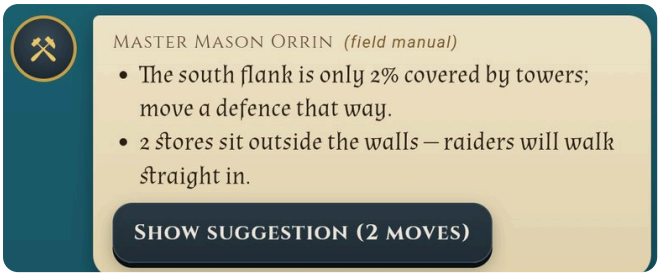
Close-ups from the app. These were taken on an offline run, so every bubble comes from the officers' built-in rules. With the server and model running, the same bubbles are filled by Qwen. Notice that every number they quote comes from the game.



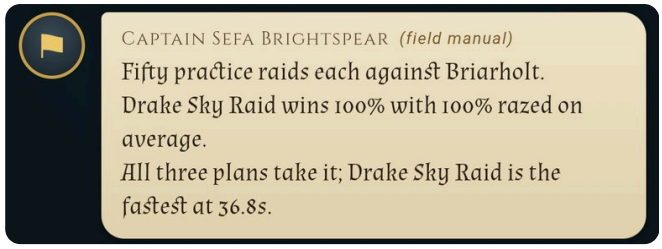
The General (here "Brannoc") gives the morning briefing on the home screen.



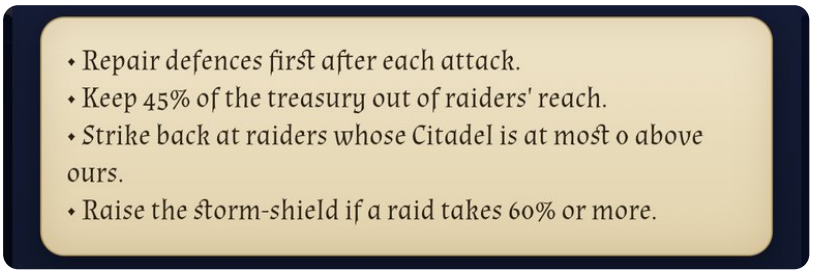
Whisper reports on a raid: MVP and the key moments, taken from the battle's event log.



Master Mason Orrin reviews the layout and offers a 2-move fix.



Captain Sefa sums up 150 practice raids and picks the fastest plan.



Watch-Captain Hale turns the night-order sliders into plain-language rules before you seal them.

CHAPTER 8

Night Defence

Orders, not trust

At night, rival lords send small **raiding parties** (troop space of 4 + 2 × their Citadel level, and no General) against your island. You are asleep, so your officers act for you, but only inside the orders you sealed.

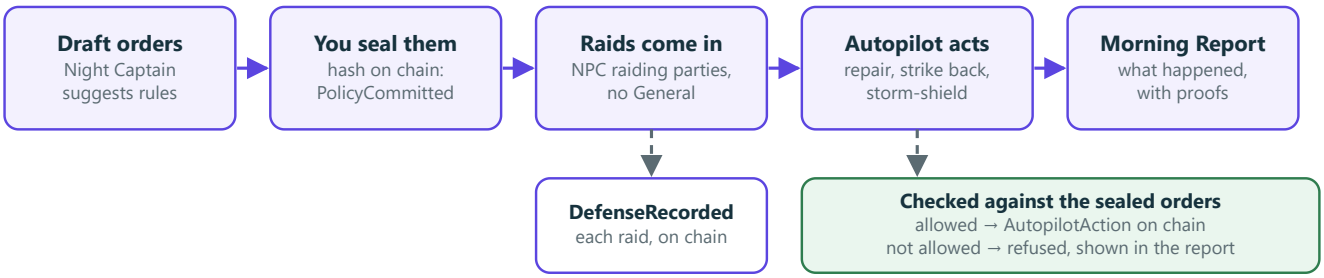
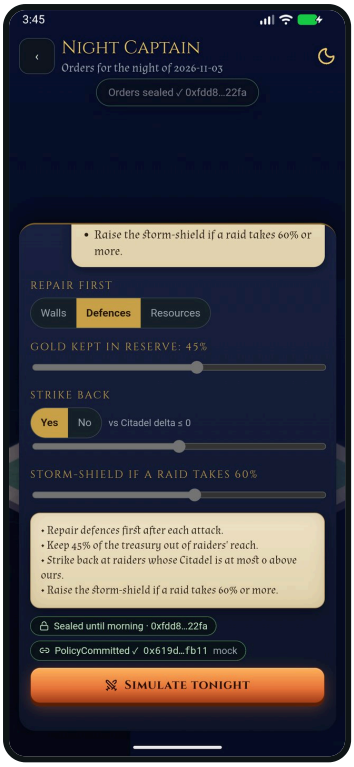
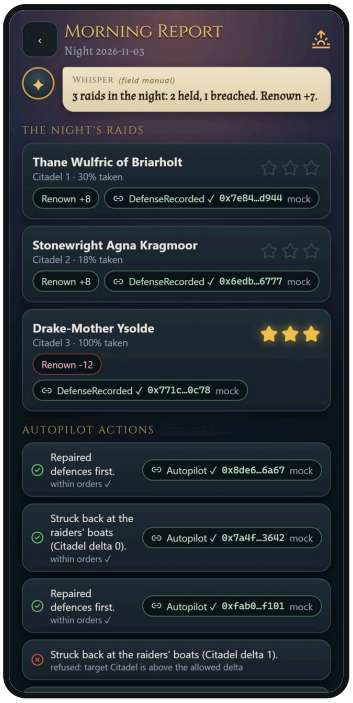


Figure 8. A night. The orders' hash goes on chain *before* the night. Each autopilot action carries that hash, and the contract only accepts actions logged against the orders you committed.



09 Night orders
Repair first: walls, defences or resources. Gold kept in reserve: 45%. Strike back: yes. Storm-shield at 60%.



10 Morning Report
Each raid with stars and a DefenseRecorded proof. Each autopilot action is marked "within orders ✓".

The report also shows actions that were **refused**. For example, "Struck back at the raiders' boats (Citadel delta 1)" is refused because the orders only allow striking back at raiders whose Citadel is at most 0 levels above yours.

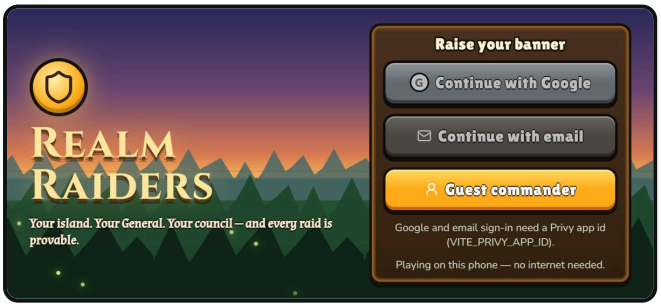
From the report, the Architect can apply a layout fix in one tap. The base is then "unsealed" until you seal it again.

> In the live run, the night produced **2 DefenseRecorded** and **4 AutopilotAction** transactions on Monad (see the table in chapter 6).

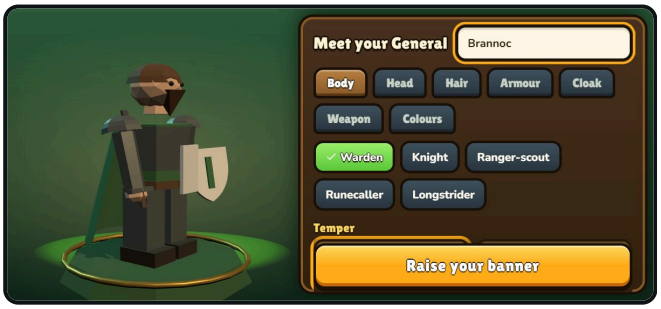
CHAPTER 9

The phone app, screen by screen

The app has 17 screens, numbered in the order a new player meets them, and it plays in **landscape** (a portrait browser is asked to rotate). The pictures below come from the app in a 20:9 phone-sized window; `pnpm screens` retakes them at 16:9, 19.5:9 and 20:9 and fails if anything scrolls sideways.



01 Sign in
Google, email, or *Guest commander*; the note says whether you are online or in the offline realm.

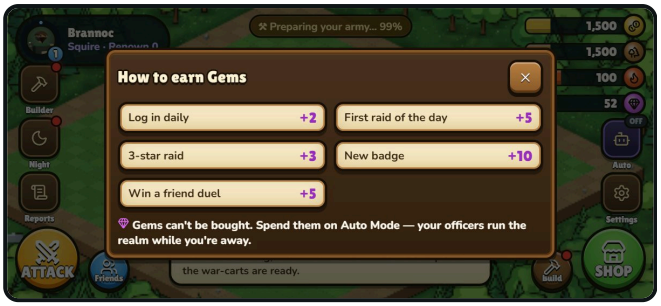


02 Raise your General
A 3D figure you style — temper, banner, look and name — then registered as an ERC-8004 agent.



03 Home island

Badge top-left, resources and Gems top-right; Builder, Night and Reports on the left, Auto and Settings on the right; ATTACK and Friends bottom-left, Build and Shop bottom-right.



03b How to earn Gems

"Gems can't be bought." The only way to get them is to play.



04 Build

Tap a building to upgrade it, long-press to move it. *Walls*: drag a finger to paint a wall line. *SAVE BASE* puts the layout's fingerprint on chain.



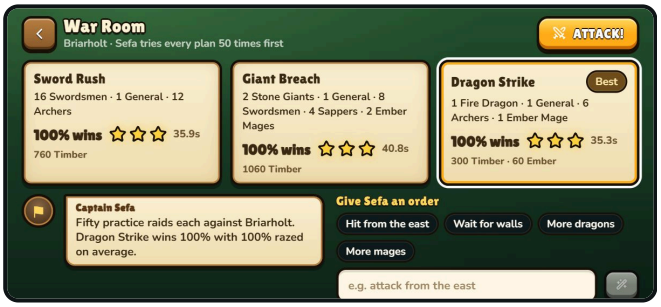
04b Shop

Defences, Resources, Army and Walls & Traps, each with a picture, price and "Built 1/2"; tap one, place the ghost, ✓.



05 Find a target

The rival's base fills the screen so you can scout it; arrows go to the next rival (friends on your Wi-Fi too), then ATTACK.



06 War Room

Sefa tries every plan 50 times first: each card shows its win %, stars, time and cost; give Sefa an order to change the plan.



07 Battle

3D-baked troops face their path; fire cones, fireballs, fist slams and arrow trails. Autoplay or take control.



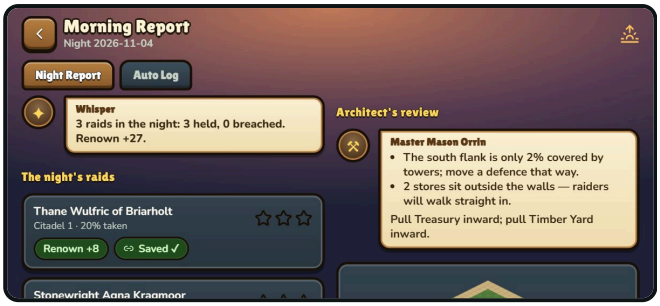
08 Result

Stars, loot and Gems earned, Whisper's report, and "Fair play verified ✓" with *How we know* for the proof.



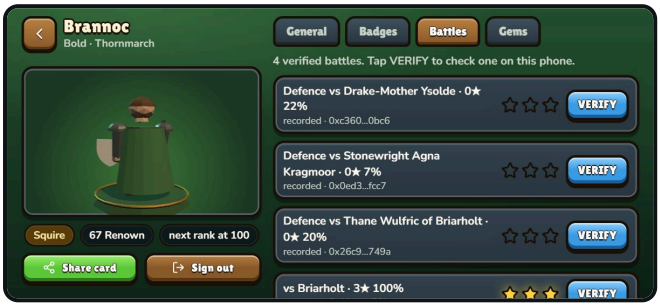
09 Night Captain

Draft, adjust and save tonight's orders; *Simulate tonight* plays the night now.



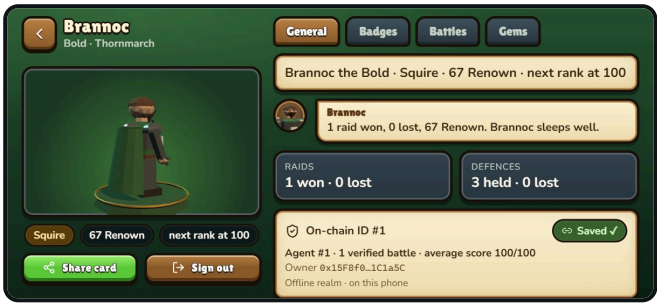
10 Morning Report

The night's raids and autopilot actions — and a new Auto Log tab.



12b Battles

Every raid with its stars; VERIFY re-runs it on the phone and compares it with the chain.



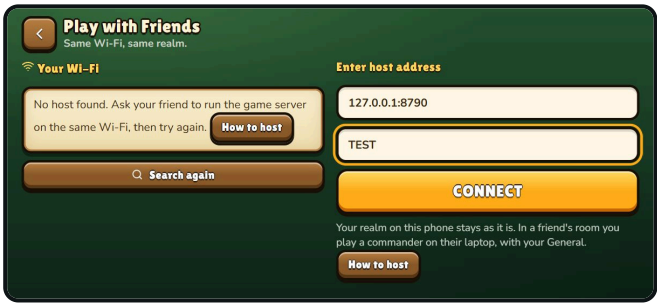
12 Profile

Rank, badges, ERC-8004 identity and reputation, battles with VERIFY.



12c Gems history

Every change to your Gems: "+5 First raid today · −4 Auto Mode 1 hour".



13 Play with Friends

Searches your Wi-Fi for a host, or type its address and the room code.



14 Lobby

Who is in the room, status chips, READY, emotes, and RAID FRIEND · LIVE DUEL.



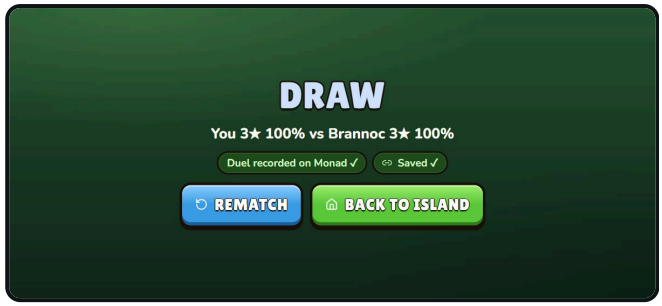
15 Live Duel

Both bases sealed, one shared seed locked, 3 minutes: pick a plan against your friend's base.



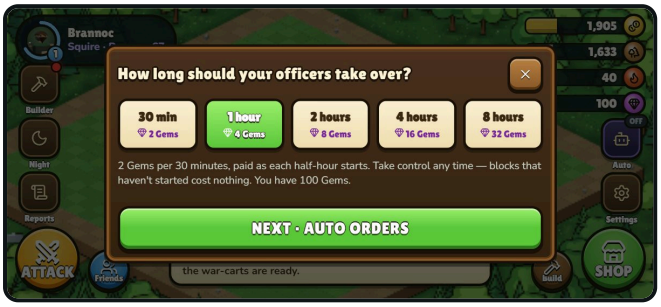
15b Duel battle

The mini-bar shows both commanders' stars and % live.



16 Duel result

Both replays re-simulated by the host; the result is recorded on RealmDuels.



17 Auto Mode: how long?

30 minutes to 8 hours, 2 Gems per half-hour.



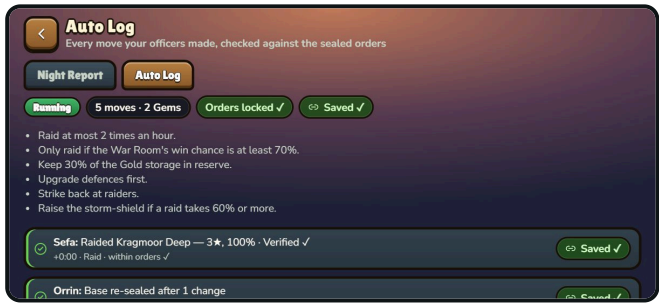
17b Auto Orders

Drafted by Watch-Captain Hale, editable, then sealed on chain like night orders.



17c Auto Mode running

Glowing border, the countdown, the live action feed and a big TAKE CONTROL.



17d Auto Log

Every move, ✓ within orders or ✗ refused, each with its Ledger entry.

How the screens are made

- **React 19** draws the menus, panels and chat bubbles. A small store holds the player's state and refreshes the home screen every 8 seconds.
- **Phaser 4** draws the island and the battle in *isometric* view (the diamond-shaped 2.5D look). Buildings, troops, arrows and effects are painted with code on a canvas.
- The battle screen plays the engine's ticks as animation. The real simulation runs in a **Web Worker**, so the drawing stays smooth.
- Every "✓" chip (for example *SeedLocked ✓0xf180…8f8b*) is a link to the transaction on the explorer. In offline mode it shows "mock".
- The fonts are **Lilita One** (buttons and titles) and **Nunito** (text), with **Cinzel** kept for the logo; all are bundled open-licence fonts, so the app needs no internet for them.
- Troops and buildings are **Three.js** models baked on the phone into sprite sheets (5 directions mirrored to 8, idle/walk/attack/death), cached in IndexedDB; without WebGL the game keeps its 2D drawings.

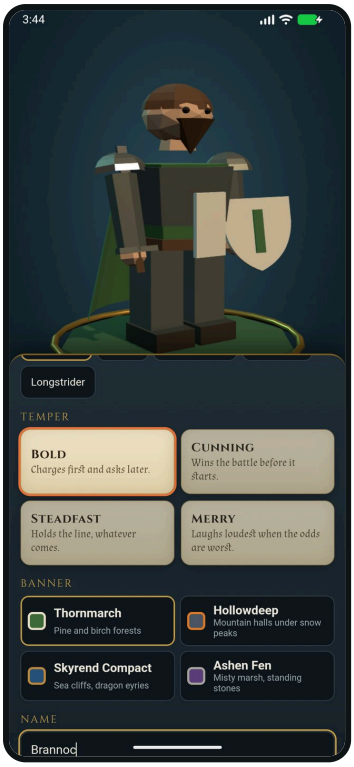
Small things that make it feel like a game

- Coins float up when you collect, with a sound.
- Stars "pop" in one by one on the result screen.
- Officers show a typing shimmer while they think.
- A red dot on an officer means they have news.
- If the server is slow, a "Courier delayed" banner appears instead of an error.
- A *Share card* on the profile makes a picture of your General and record to share.

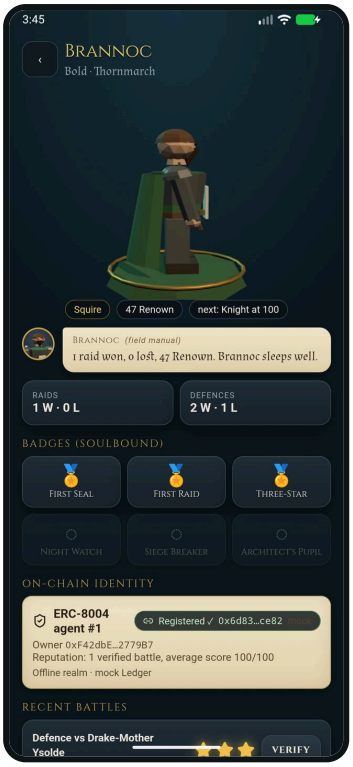
CHAPTER 10

The 3D General, art and sound

No art was downloaded or bought. Every picture in the game is **generated by code** when the app runs. This keeps the game original and the app small (5.6 MB).



CREATOR
A 3D figure on a turntable, rebuilt live as you change options.



SAME GENERAL, PROFILE
The figure spins on its plinth above your rank and badges.

How the General is made

1. **Built from blocks.** Three.js assembles head, body, cloak, shield and weapon from simple shapes. Your choices (body, head, hair, cloak, weapon, shield, colours and faction) are stored as a small checked "appearance" record.
2. **Fingerprinted.** The appearance is hashed and stored with the General, so the same record always rebuilds the same figure.
3. **Portrait painted.** The 3D figure is rendered once to a square image, used in the officer bubbles and the share card.
4. **Battle sprite baked.** The figure is also rendered small from the battle camera's angle, so the General in the 2D battle matches your 3D one.

The rest of the art

- **Island and battle:** buildings, walls, troops and effects are drawn with Canvas 2D shapes, in the colours of your faction.
- **Landing page:** a 3D island with a castle, trees and a glowing sea.
- **Sound and music:** taps, deploys, shots, explosions, coins, building, stars and the victory tune are made live with the Web Audio API, and so are two original background tunes (a calm one at home, a drum-driven one in battle; on/off in Settings). The APK ships no audio files.
- **App icon:** a vector "adaptive icon" for Android.

> An **originality check** (`pnpm check:originality`) scans all 285 source files for names, terms and asset paths from other games and fails the build if it finds any. The world, names and look are original to Realm Raiders.

CHAPTER 11

Online, offline and Android

Two ways to play

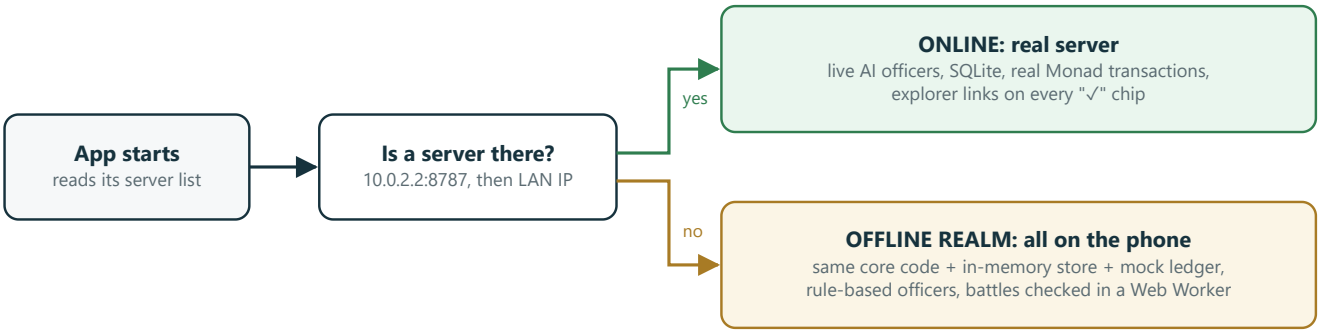


Figure 9. Choosing a mode. The app tries each server address in its list. If none answers, it starts the offline realm, using the same game-services code as the server.

Because `packages/core` runs on the phone too, the offline realm is the *real* game, not a demo: the same rules, the same economy, and the same raid checks. It only swaps the database for an in-memory store (saved on the phone), the blockchain for a mock ledger, and the AI model for the field manuals.

Turning the web app into an Android app

- **Capacitor 8** wraps the built web app in a native Android shell (a *WebView*), giving a normal installable **APK**.
- Portrait only, single window, an adaptive vector icon, and **App Links**, so links to the website can open the app. The website publishes an `assetlinks.json` file with the fingerprints of the signing keys.
- Two builds: **debug** (7.1 MB, may talk to a plain-http server on the PC for testing) and **release** (5.6 MB, signed with a private release key kept outside the project folder).

Testing on a real Android system

A script (`e2e-android/apk-playthrough.ts`) uses **Playwright** to drive the **installed APK** on the Android emulator, through the app's own WebView: sign in → create General → seal base → War Room → raid → verified result → night orders → morning report → Auto Mode → profile. It took the screenshots in this report and checks that no page errors happen. It passed **live** (against the server and Monad) and **offline**.

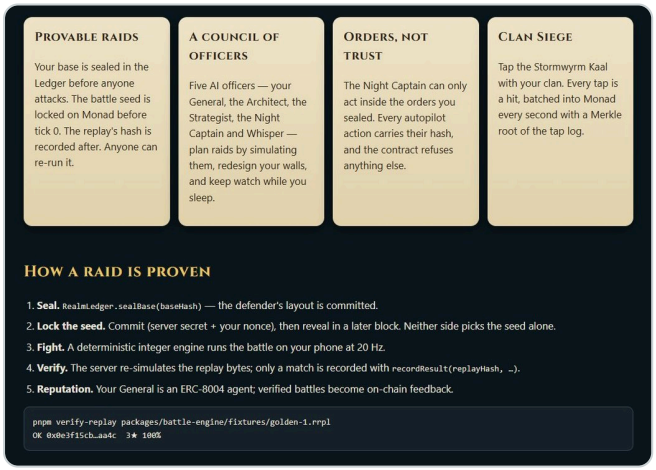
> **Problems found only on the phone, and fixed:** Android blocked plain-http calls from the app (allowed in debug builds only). The server refused the phone's `PUT` requests until its CORS settings listed them (a test now covers it). The WebView quietly switched image links to https, so images are now loaded as blobs. Portrait links pointed to "localhost", which is the phone itself, so they are now rewritten to the server's address.

CHAPTER 12

The landing website and tools



The landing page (`apps/landing`): a Three.js island under a night sky, with the download button and a "How it works" link.



Further down: four feature cards and the 5 steps of "How a raid is proven".

Also on the site

- `/.well-known/assetlinks.json` for Android App Links.
- `/redirect`, the page the Google/email login comes back to, which reopens the app.
- A slot for a trailer video, empty for now.

Tools

- **verify-replay**: re-checks a replay file or a battle ID, optionally against the chain.
- **live-run**: plays every on-chain moment through the API and prints explorer links.
- **digest** and **make-golden**: produce the determinism fingerprint and the golden battles.
- **assets/import**: ready for adding free (CC0) art packs later.

```
# anyone can re-check the first golden battle
pnpm verify-replay packages/battle-engine/fixtures/golden-1.rrpl
OK 0x0e3f15cb…  3★ 100%
```

CHAPTER 13

How it was built

The work followed the project guide (`main_project/Project_Guide.md`), which splits the game into **12 phases**. Each phase ended with working, tested code before the next began. After phase 11 came the go-live step: deploying to Monad and play-testing the APK.

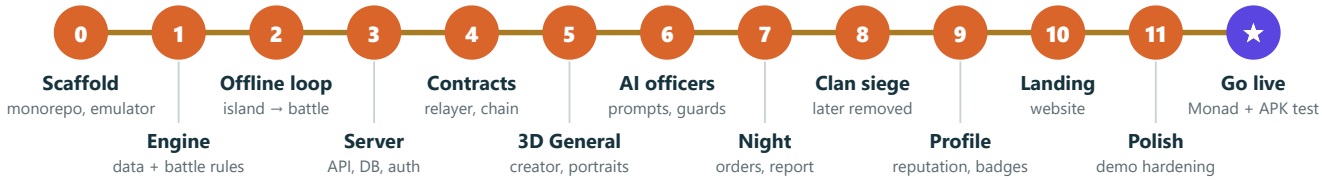


Figure 10. The build order. The engine came first because everything else depends on it. The chain was added after the game already worked offline.

The building blocks used

Tool	Used for
TypeScript 6, pnpm 10, Node 22	The language, the package manager and the runtime for everything except the contracts.
Vite 8, Vitest 5	Building the app and website quickly; running the 140 unit tests.
React 19, Phaser 4, Three.js	Screens, the 2D isometric island and battle, the 3D General and boss.
Capacitor 8, Android SDK, Gradle	Packaging the app as an APK and signing it.
Fastify 5, SQLite, Drizzle, jose	The server, its database, and login sessions (signed tokens).
viem	Talking to Monad: sending transactions, reading events, deploying.
Solidity, OpenZeppelin, solc-js, ethereumjs VM	Writing, compiling and testing the smart contracts.
Ollama + Qwen3 4B	The local language model behind the officers.
Playwright	Automatic play-throughs in Chrome and on the Android emulator.

How each phase was done

1. **Write the numbers first** (game-data), then the rules (engine), with tests for every rule: walls block, sappers break walls, drakelings fly over, traps trigger, the rally works.
2. **Lock the rules down** with golden replays and the 200-battle fingerprint, so later work can't change battles by accident.
3. **Build the game loop offline**, then move the same services to the server and add the database and logins.
4. **Add the chain behind a switch**: a mock chain for development and tests, and the live chain with a real relayer for Monad.
5. **Add the AI last**, with a rule-based field manual written first, so the game never depends on the model being up.
6. **Play it on a phone**, fix what breaks there, deploy, and play again live.

Problems met along the way, and the fixes

Problem	Fix
Battles first took 30–40 ms each, too slow for 150 practice battles.	Added a "can walk here" grid and a faster queue for path-finding, and let troops spread apart only every other tick. Now 12–24 ms.
The fingerprint test caught the clock (`Date`) being used in the digest script.	Removed it. This is exactly what the "banned functions" test is for.
Night raiders 3-starred every starter base.	Raid armies could only grow, never shrink, to fit the attacker. Now they scale both ways, night raiders come without a General, and the starter base got its two caltrop traps.
The local AI took about 10 seconds per answer.	Turned off its "thinking" step and moved from the 8B to the 4B model: about 2 seconds.
The AI sometimes invented numbers, or said a plan "stalls at 100%".	Added the number-grounding check and a contradiction check. Failing answers fall back to the field manual.
On the phone, the game canvas filled only a quarter of the screen.	Made the canvas fit its container with CSS, and moved the sea background onto the canvas itself.
The 3D boss view didn't appear on first open.	Made its container always render, and fixed the camera framing.
The Foundry toolkit couldn't be installed (blocked on this PC).	Compiled with solc-js, tested on an in-memory EVM, and deployed with a viem script instead.
Android-only issues: http blocked, PUT refused, images upgraded to https, "localhost" portraits.	Debug-only mixed content, explicit CORS methods (with a test), blob-loaded images, and portrait links rewritten to the server's address.
An old resize listener crashed after leaving a screen ("reading 'setZoom'").	The listener is now detached when the scene shuts down, and guarded.

Keeping secrets safe

The relayer's private key, the session secret and the release-signing password live in files that git ignores (`.env` , `keystore.properties`), and the release key file sits outside the project folder. A **pre-commit guard** refuses any commit that contains them. None of them appear in this document.

CHAPTER 14

Proof that it works

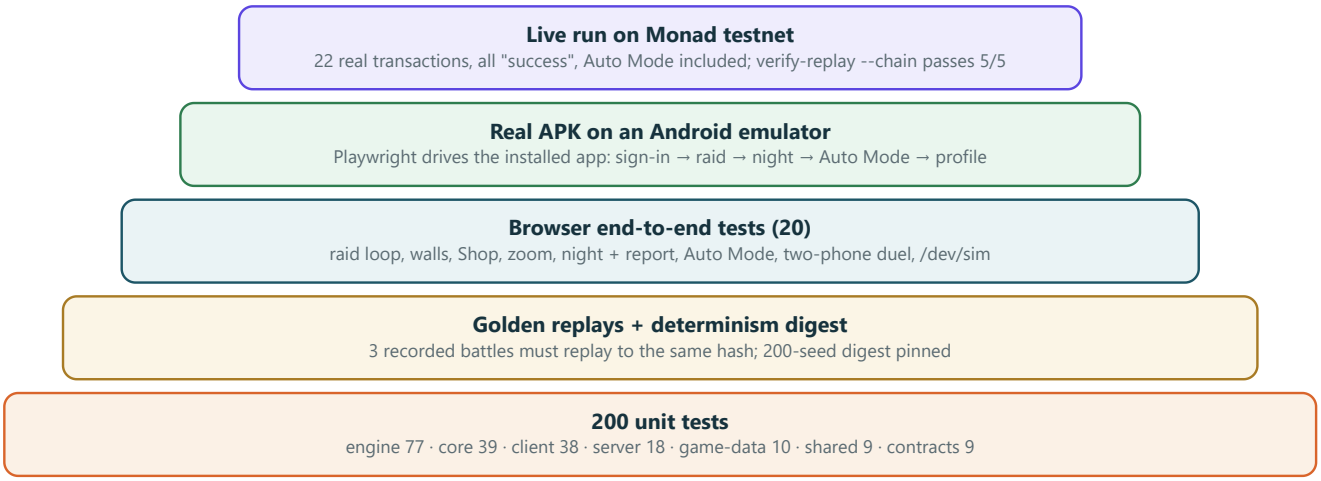


Figure 11. Layers of testing, from small and fast (bottom) to real-world (top).

Check	Result
Unit tests (Vitest)	**200 passed**: battle-engine 77, core 39, client 38, server 18 (+1 that needs a local chain and is skipped), game-data 10, shared 9, contracts 9
Browser end-to-end (Playwright)	**20 passed**: full raid loop; wall brush; Shop placement; zoom range; build + night + report; Auto Mode; two-phone friend raid and Live Duel; profile; /dev/sim digest
Determinism fingerprint	`0xc0afd3965bc7fff416f2285bea70284ee39fd4dd6a88ed8a996e8f2be1037ebe`, identical in Node 22, Chrome, and the Android WebView
Golden replays	**3 / 3 OK** on all three platforms
Type checks	**clean** in every package (`pnpm -r typecheck`)
Originality gate	**passed** over 285 files
Contracts on Monad testnet	**3 deployed**, source verified ("perfect" match)
Live run on Monad	**22 / 22 transactions succeeded**: agent registration, seal, commit, seed, result, reputation, 2 badges, night orders, 3 defences, 4 autopilot actions, Auto Orders and 5 Auto Mode moves
verify-replay --chain	**5 / 5 checks** on a live raid
APK on Android emulator	**played start to finish**, live and offline, with no page errors
AI officers (live)	Answers in **1.8–3.2 s**; the built-in rules take over cleanly otherwise
Performance	12–24 ms per battle; 150 War Room battles in about 1.3 s

> **Is the APK playable?** Yes. It installs and plays the whole loop on Android: on its own (offline realm), or connected to the server, where the raids, badges and reputation become real Monad transactions.

CHAPTER 15

What is not done yet

These were skipped by choice or blocked by the environment. None of them stops the game from being played.

Item	Status and what it needs
Google / email sign-in (Privy)	The code is in place but no Privy app was created, so the app uses *Guest commander*. It needs a Privy app ID and secret in the settings.
Foundry `forge test`	The installer was blocked on this PC. The same contracts are tested on an in-memory EVM instead, and the Foundry test file is ready to run.
Public server	The server runs on this PC. Phones reach it on the same Wi-Fi or from the emulator. Release builds need an **https** address (for example a Cloudflare tunnel) before they can play online, and agent cards need `PUBLIC_BASE_URL` set to a public host.
Hosted AI model	No cloud API key is set; the local Qwen3 4B is used. Add `LLM_BASE_URL` and `LLM_API_KEY` to use a hosted Qwen first.
Live-chain run of friend play	Auto Mode now runs on the live chain (Auto Orders and 5 moves in the 2026-11-04 live run). Friend raids and Live Duels were tested on the mock ledger, which follows the same rules. RealmDuels is deployed, verified, and its relayer role was read back on chain.
Two real phones	Play with Friends was tested with an emulator against a browser and with two browsers; not yet on two physical phones on one Wi-Fi.
Art packs and trailer	All art is made by code (2D drawings and 3D models baked on the phone). The landing page has an empty trailer slot.
Publishing the code	The work is saved in local git commits (one small commit per change) and hasn't been pushed anywhere.

Small differences from the guide

- Built in a new folder, `realm-raiders/`, with an extra `packages/core` so the APK can play fully offline.
- Node 22 and pnpm 10 (the guide suggested Node 24 and pnpm 12).
- Starting ember is 100, all troop types are available from the start (the Muster Hall sets their level), and loot scales with destruction.
- Replays carry the sealed base, and RealmLedger has an extra `recordDefence` for night raids.
- The local model is **Qwen3 4B** rather than 8B, to fit the 4-second budget on a 6 GB laptop GPU.

CHAPTER 16

How to run it, and a glossary

```
# one-time setup
pnpm install
# play in the browser, whole game on-device (mock ledger) → http://localhost:5173
pnpm dev:local
# app + server (live Monad + live officers need: ollama serve)
pnpm dev
# build the Android debug APK
pnpm apk
# all checks
pnpm -r typecheck; pnpm -r test; pnpm e2e; pnpm check:originality
# every on-chain moment, end to end, with explorer links
pnpm tsx tools/live-run.ts
# rebuild this document
node docs/explainer/build.mjs
```

Glossary

Blockchain: a shared public record that many computers keep in sync. Once something is written, it can't be quietly changed.

Monad testnet: the practice version of the Monad blockchain. Its coins have no money value, so it's safe for testing.

Smart contract: a small program that lives on the blockchain and follows its code exactly.

Transaction: one action sent to the blockchain, such as "record this result".

Gas: the small fee a transaction costs. The game's relayer pays it, not the player.

Relayer: the server's wallet, which sends transactions for players.

Hash: a short fingerprint of any data. Change one bit of the data and the fingerprint changes completely.

Seed: the starting number for the game's dice. Same seed, same rolls.

Commit–reveal: first publish the hash of a secret, then reveal the secret later. It proves you didn't change your mind.

Deterministic: always giving the same output for the same input, on any device.

Replay: a tiny recording of a battle's inputs (what was tapped and when), not a video.

Tick: one step of the battle clock, 1/20 of a second.

ERC-8004: a standard for registering AI agents on a blockchain, with an ID card and a reputation score.

Soulbound badge: a token that can't be sold or given away; it stays with the player who earned it.

Merkle root: one fingerprint that stands for a whole list, so any single entry can be checked later.

Sourcify verification: proof that the published source code is exactly what runs at a contract's address.

Monorepo: one repository holding several packages that share code.

API: the set of web addresses the app calls to ask the server for things.

WebSocket: an always-open connection, so the server can push updates instantly.

Web Worker: a background thread in the app, used to run battles without freezing the screen.

APK: the install file for an Android app.

Capacitor: a tool that wraps a web app into a native mobile app.

LLM: a large language model, the kind of AI that writes text. Here: Qwen3.

Ollama: a program that runs language models on your own computer.